MLOps Experimental Learning Assignment · Total marks 50

End-to-End ML Model Development, CI/CD and Production Deployment: Heart Disease Risk Classifier (UCI Cleveland)

Student name	SHIVARAJA C M
Student number	2025AE05109
Email	2025AE05109@wilp.bits-pilani.ac.in
Course	MLOps — AIMLZG523 (S1-26), BITS Pilani Work Integrated Learning Programmes
Contribution	100 % (individual assignment)
Code repository	`https://github.com/shivaraj17cm/heart-risk-mlops` (public)
Demo video (8 min 52 s)	`https://drive.google.com/file/d/1cYS4DSD7XL4Otd4_b4uulv6-IkJaSwq9/view` (Google Drive, public link)
API access	Local Kubernetes (Docker Desktop): `http://localhost:8080/predict` — access instructions in §11, full setup in §14; no public cloud endpoint was created.

Abstract

This report documents a reproducible MLOps pipeline that predicts the presence of heart disease from the 13 clinical attributes of the UCI Heart Disease (Cleveland) dataset. Data acquisition, EDA, training and experiment tracking were executed in the BITS **Prayogshala** virtual lab; the code, data, model and evidence are versioned on GitHub; a four-stage GitHub Actions pipeline lints, tests, retrains and container-tests every change; the packaged model is served by a FastAPI service in a Docker image, deployed with two replicas on Kubernetes behind a LoadBalancer, and monitored with Prometheus and Grafana. Three tuned classifiers were compared with stratified 5-fold cross-validation; **Logistic Regression** was selected (CV ROC-AUC 0.907 ± 0.019; hold-out ROC-AUC 0.959, recall 0.964). The same metrics were reproduced independently by CI on a different Python version.

1. Objective and problem statement

Objective: design, develop and deploy a scalable, reproducible ML solution using MLOps practices — automation, experiment tracking, CI/CD, containerisation, deployment and monitoring. Problem: build a binary classifier that estimates the risk of heart disease (target = presence/absence) from patient attributes and expose it as a cloud-ready, monitored API. *The model is an academic exercise and not a clinically validated diagnostic tool.*

Deliverables and where they are

Deliverable (assignment)	Location / evidence
Code, Dockerfile, requirements	`src/heartrisk/`, `Dockerfile`, `requirements.txt`, `requirements-serve.txt`
Cleaned dataset + download script	`data/processed/heart_clean.csv`, `scripts/download_data.py`, `data/raw/manifest.json`
Notebooks / scripts (EDA, training, inference)	`notebooks/01_eda, 02_training_mlflow, 03_inference.ipynb` (executed), `python -m heartrisk.{data,eda,train,predict}`
Unit tests	`test/` — 53 Pytest tests, 95 % line coverage
CI/CD workflow	`.github/workflows/ci.yml` — every push run green (#1–#4, #6 onward); PR run #5 deliberately red
Deployment manifests	`deploy/k8s/*.yaml`, `deploy/kustomization.yaml`, monitoring configs in `deploy/monitoring/`
Screenshots	`screenshots/` — 36 captures, indexed in `screenshots/README.md`
Report, video	This 10-page PDF; pipeline demo video (Google Drive link above)

Key results. 303 patients, 13 features, 6 rows with missing values (imputed in-pipeline). Selected model: Logistic Regression (C = 1.0, L2, class-weight balanced). Hold-out test (n = 61): accuracy 0.885, precision 0.818, recall 0.964, ROC-AUC 0.959. Served at 2 replicas on Kubernetes; logged `/predict` latencies of 9–27 ms; 547 predictions monitored in Grafana.

2. Architecture and workflow

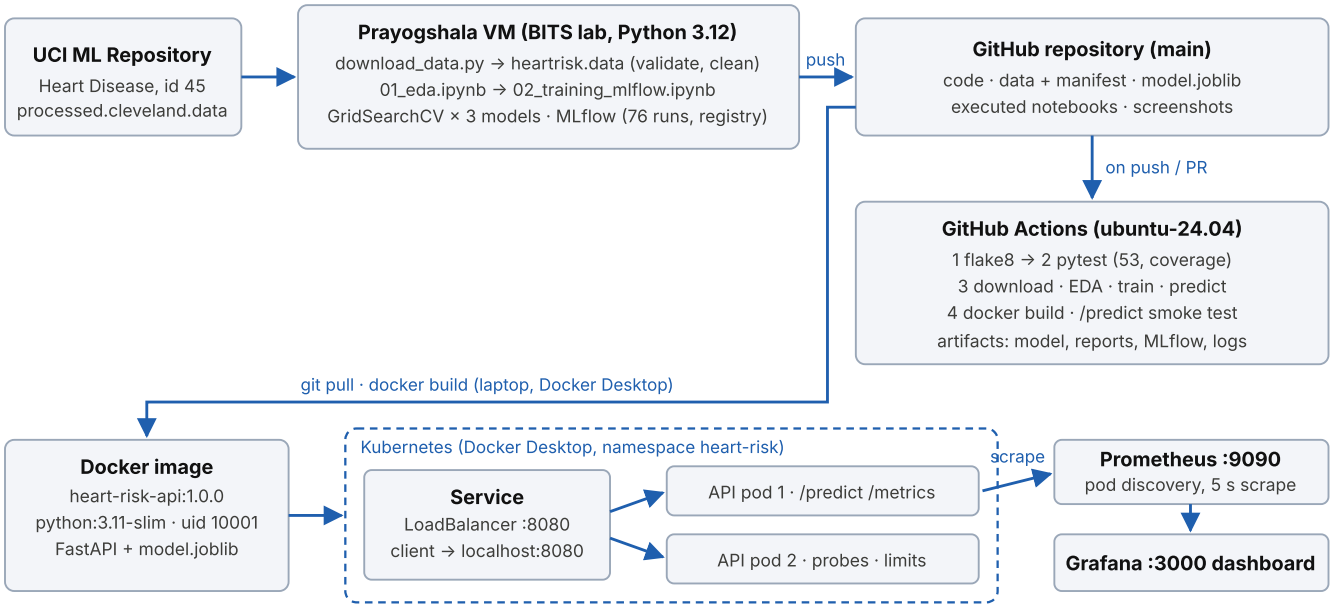


Figure 1. System architecture: lab execution (Prayogshala) → versioned repository → CI/CD → container → Kubernetes → monitoring.

Technology choices and trade-offs

Concern	Choice	Why, and the alternative considered
Modelling	scikit-learn Pipeline	One serialisable object holds feature engineering, imputation, scaling, encoding and the classifier, so training and serving cannot drift and preprocessing is fitted on training folds only (no leakage). Alternative: separate pre-processing scripts — rejected (train/serve skew).
Tracking	MLflow 2.22 (SQLite + registry)	Named in the assignment; parent/child runs make every grid point auditable; registry alias `champion` identifies the deployable version. W&B would need an external account.
Serving	FastAPI + Uvicorn	Pydantic validation produces precise 422 errors and an OpenAPI page for free; async-ready. Flask would need manual validation.
CI/CD	GitHub Actions	Repository-native, free for public repos, per-run artifacts and job summaries. Jenkins needs a server to operate.
Deployment	Kubernetes on Docker Desktop	Allowed by the assignment (local Kubernetes); zero cost, same manifests run on GKE/EKS/AKS. A public cloud cluster was not created to avoid cost and public exposure.
Monitoring	Prometheus + Grafana	Pull-based metrics with Kubernetes pod discovery (every replica scraped, not just one behind the Service); Grafana dashboard provisioned as code.

Execution environments

Environment	Used for
Prayogshala VM	Data acquisition, validation, EDA notebook, training + MLflow UI, pytest, flake8 (host `ip-10-3-24-59.ap-south-1.compute.internal`, user s2025ae05109, Python 3.12.9). Results committed from the VM (commits 2ce748b, 91ca773).
GitHub Actions	Independent re-execution of lint, tests, training and container smoke test on every push (Python 3.11).
Laptop (Windows, Docker Desktop 29.3, Kubernetes v1.34.1)	Docker build/run, Kubernetes deployment, Prometheus/Grafana monitoring.

3. Data acquisition and cleaning

Source. UCI Machine Learning Repository, *Heart Disease* (dataset id 45, CC BY 4.0), Cleveland file `processed.cleveland.data`: 303 rows × 14 columns, no header, missing values encoded as `?`. The Cleveland subset was chosen because it is the version used in the original studies and is the one named "UCI"; the widely circulated Kaggle *heart.csv* (1,025 rows) was not used because it consists largely of duplicated records with re-coded categories.

Reproducible acquisition (`scripts/download_data.py`): direct file URL with fallback to the official ZIP, three retries with back-off, structural validation (14 fields per row), and a manifest recording URL, size, row count, timestamp and SHA-256 (`a74b7efa…0427c8`). Re-runs reuse the verified copy (idempotent) and warn if the upstream file ever changes. The run in Prayogshala is shown in screenshot 02.

Validation and cleaning (`heartrisk.data`): column set and numeric types are asserted; codes are checked against the UCI documentation (e.g. `cp ∈ {1..4}`, `thal ∈ {3,6,7}`) and continuous values against plausible ranges; exact duplicates are removed; the diagnosis `num` (0–4) is binarised to `target = 1 if num > 0`. Result on the real file: 0 duplicates, 0 invalid rows, 6 rows with missing values (`ca`: 4, `thal`: 2), class balance 164 absent / 139 present (45.9 % positive). Missing values are deliberately *not* filled at this stage: median / most-frequent imputation is learned inside the model pipeline from training folds only, so no statistic from validation or test rows leaks into training.

Group	Features	Treatment in the pipeline
Numeric / count	age, trestbps, chol, thalach, oldpeak, ca	median imputation → StandardScaler
Binary	sex, fbs, exang	most-frequent imputation (already 0/1)
Nominal categorical	cp (4 types), restecg (3), slope (3), thal (3)	most-frequent imputation → OneHotEncoder(handle_unknown="ignore")
Engineered (tunable)	hr_pct_predicted = thalach / (220 − age)	numeric branch; kept or dropped by grid search

Leakage and integrity checks. The only target-derived column, `num`, is removed after binarisation; no feature is computed from the label; the train/test split is stratified and seeded; the same pipeline object is cross-validated, packaged and served.

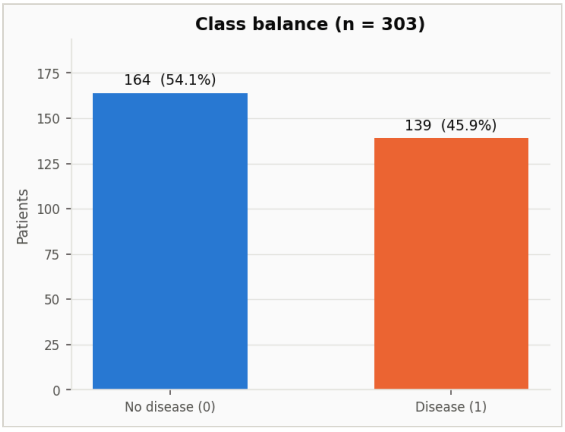


Figure 2. Class balance after binarisation (real data).

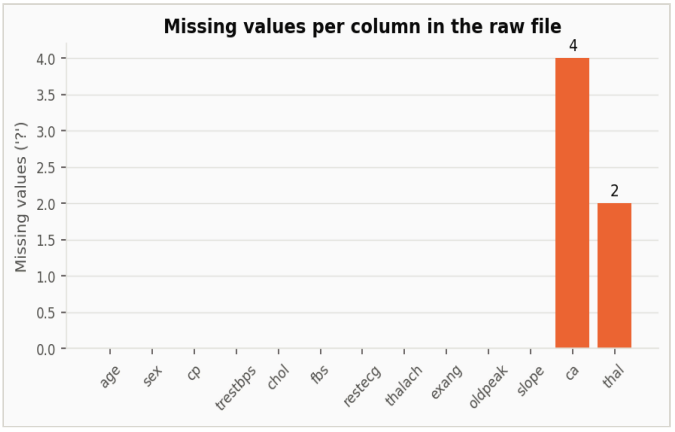


Figure 3. Missing values per column in the raw file.

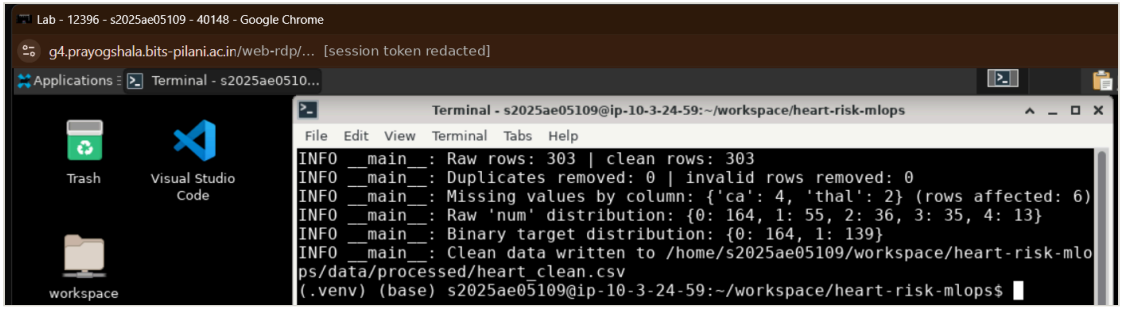


Figure 4. Validation and cleaning executed in the Prayogshala VM — excerpt of screenshot 03 (VM bar, host and output; session token redacted).

4. Exploratory data analysis

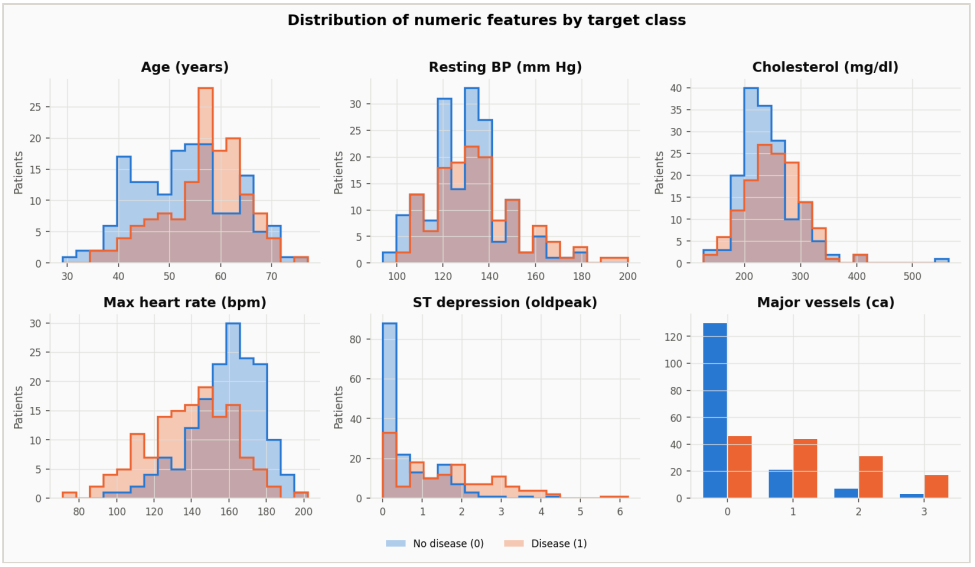


Figure 5. Histograms of numeric features by class (blue = no disease, orange = disease).

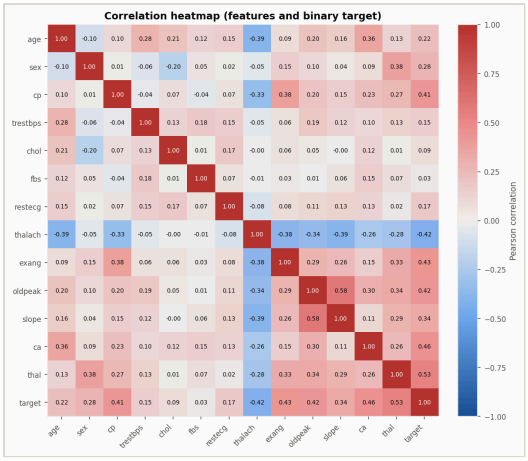


Figure 6. Pearson correlation heatmap (13 features + target).

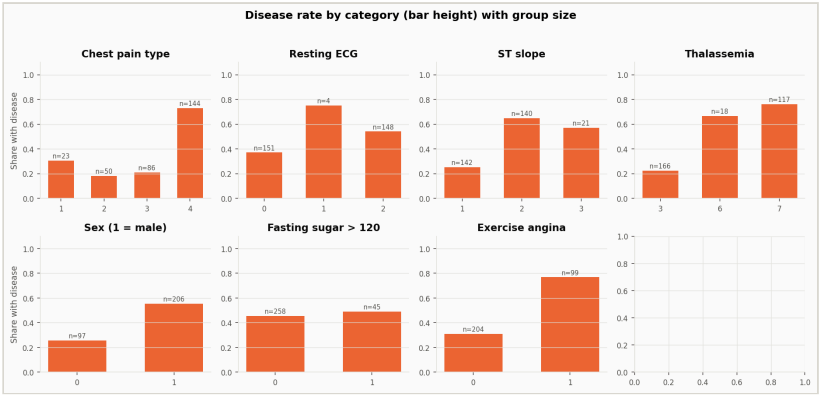


Figure 7. Disease rate per category (bar) with group size n.

Observations (computed from the data, `reports/eda_summary.json`)

- **Strongest linear associations with the target:** thal +0.53, ca +0.46, exang +0.43, oldpeak +0.43, thalach −0.42, cp +0.41; weakest: chol +0.09, fbs +0.03 — cholesterol alone barely separates the classes.
- **Class means:** patients with disease are older (56.6 vs 52.6 y), reach a lower peak heart rate (139 vs 158 bpm), show larger ST depression (1.57 vs 0.59) and more coloured vessels (1.14 vs 0.27).
- **Categorical signals:** disease rate is 72.9 % for asymptomatic chest pain (cp = 4) vs 18–30 % for other types; 76.1 % for reversible thalassemia defect (thal = 7) vs 22.3 % for normal; 76.8 % with exercise-induced angina vs 30.9 % without; 55.3 % in males vs 25.8 % in females.
- **Data quality:** one cholesterol value of 564 mg/dl is extreme but physiologically possible and was retained; classes are near-balanced, so accuracy is meaningful, but ROC-AUC, precision and recall are reported throughout.

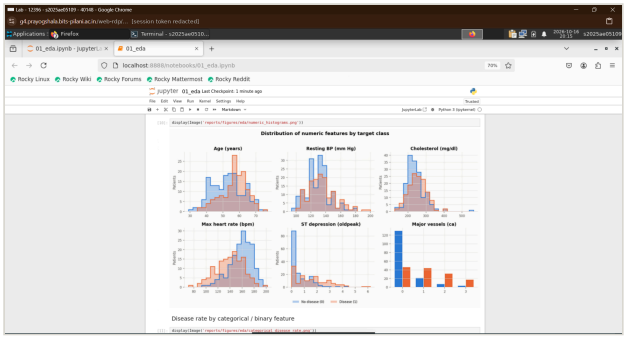


Figure 8. 01_eda.ipynb executed in Prayogshala (screenshot 04).

5. Feature engineering and model development

The data were split once into an 80 % training set (242 rows) and a 20 % **stratified hold-out test set** (61 rows, seed 42) that was used only for the final estimate. Each model family is a single scikit-learn `Pipeline` (feature builder → ColumnTransformer → classifier) tuned with `GridSearchCV` over **stratified, shuffled 5-fold cross-validation** on the training set, scoring accuracy, precision, recall, F1 and ROC-AUC, refitting on ROC-AUC. All seeds, grids and the selection rule live in `configs/train.yaml` and are logged to MLflow.

Model	Hyper-parameter grid (72 configurations, 360 fits)	Selected parameters
Logistic Regression	C ∈ {0.01, 0.1, 1, 10} × penalty {L1, L2} × class_weight {none, balanced} × add_hr_ratio {T, F} (32)	C = 1.0, L2, balanced, add_hr_ratio = False
Random Forest (300 trees)	max_depth {3, 5, None} × min_samples_leaf {1, 5} × max_features {sqrt, 0.5} × add_hr_ratio (24)	depth 5, leaf 5, sqrt, False
Gradient Boosting	n_estimators {100, 200} × learning_rate {0.05, 0.1} × max_depth {2, 3} × add_hr_ratio (16)	100, 0.05, depth 2, True

Engineered feature. `hr_pct_predicted` expresses peak heart rate as a fraction of the age-predicted maximum (220 − age), a standard exercise-testing index. It was treated as a hyper-parameter rather than assumed useful: the search kept it only for Gradient Boosting, an honest negative result for the linear and forest models, whose CV score did not improve.

Selection rule (fixed before training). Rank families by mean CV ROC-AUC; if several are within 0.01 of the best, prefer the simpler one (LR → RF → GB). The test set is never used for selection.

6. Model comparison and evaluation

Model	CV ROC-AUC	CV acc.	CV prec.	CV recall	Train−CV AUC gap	Test ROC-AUC	Test acc.	Test prec.	Test recall
Logistic Regression ✔	**0.907 ± 0.019**	**0.855**	**0.861**	**0.828**	**0.024**	**0.959**	**0.885**	**0.818**	**0.964**
Random Forest	0.901 ± 0.025	0.835	0.844	0.792	0.063	0.960	0.902	0.867	0.929
Gradient Boosting	0.876 ± 0.027	0.830	0.833	0.801	0.107	0.953	0.902	0.867	0.929

Decision. Logistic Regression has the highest mean CV ROC-AUC (0.9071) *and* the smallest train-vs-validation gap (0.024 vs 0.063 and 0.107), i.e. it generalises best; it is also the most interpretable and fastest to serve. Random Forest is 0.001 higher on the 61-patient test set, which is well inside sampling noise and, by design, not a selection criterion. On the hold-out set the selected model catches 27 of 28 disease cases (recall 0.964) with 6 false alarms (specificity 0.818) — a sensible operating point for a screening-style aid where missed cases are costlier. Test scores exceeding CV scores for all models reflect the small test set, and are reported as a limitation (§13).

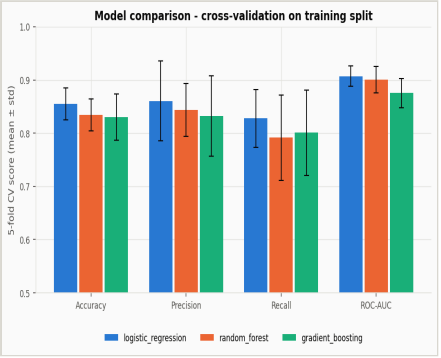


Figure 9. CV metrics (mean ± std) of the three tuned models.

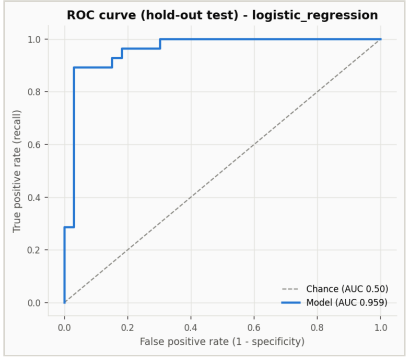


Figure 10. Hold-out ROC curve, LR (AUC 0.959).

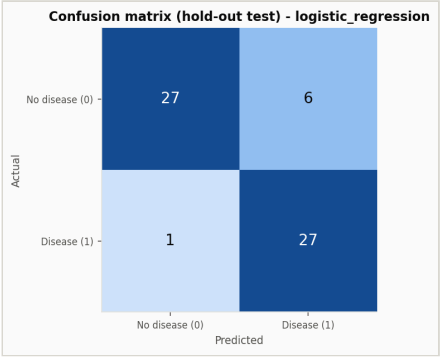


Figure 11. Hold-out confusion matrix, LR.

Interpretation (standardised LR coefficients, `feature_importance.png`): risk increases with male sex, number of coloured vessels, asymptomatic chest pain, reversible thal defect and exercise angina, and decreases with normal thal, typical angina, up-sloping ST segment and higher peak heart rate — consistent with the EDA.

7. Experiment tracking (MLflow)

Every training invocation creates, in experiment `heart-disease-uci`: one **parent run per model family** (fixed and best hyper-parameters; CV mean ± std for five metrics; per-fold ROC-AUC as a stepped metric; hold-out metrics; ROC curve, confusion matrix and coefficient/importance plots; CV results CSV; the fitted pipeline logged as an MLflow model with signature, input example, bundled code and pinned requirements), one **child run for each of the 72 grid configurations** (its parameters and CV metrics), and a **model-selection run** (comparison table and chart, rule parameters, reason text). Tags record git commit, data SHA-256, row counts, Python and scikit-learn versions and host. The selected run is registered as `heart-risk-classifier` v1 with alias `champion`. A unit test asserts that logged metrics equal the values returned by the training code.

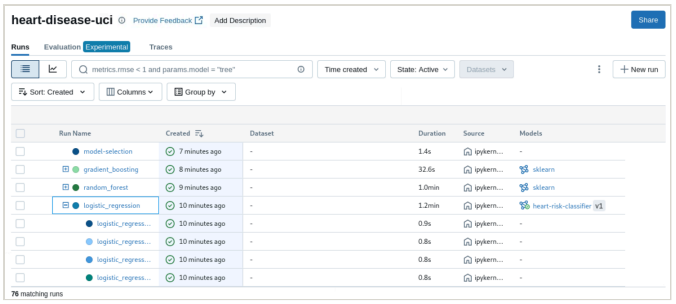


Figure 12. MLflow UI in Prayogshala (excerpt of screenshot 07): 76 runs, nested grid runs, registered model.

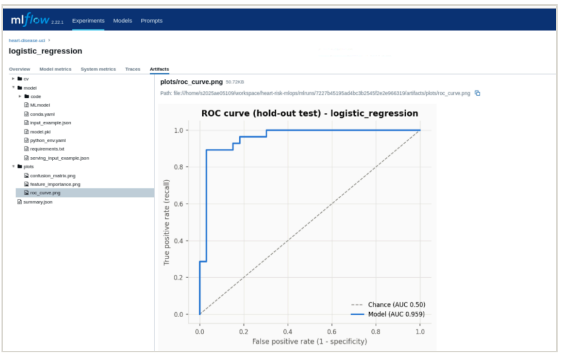


Figure 13. Logged artifacts (excerpt of screenshot 09): plots, CV results, MLflow model.

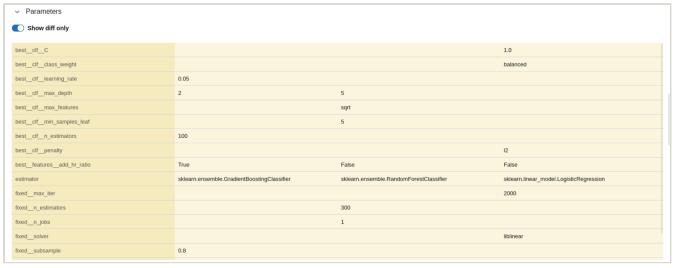


Figure 14. Run comparison (excerpt of screenshot 10): tuned parameters of the three models.

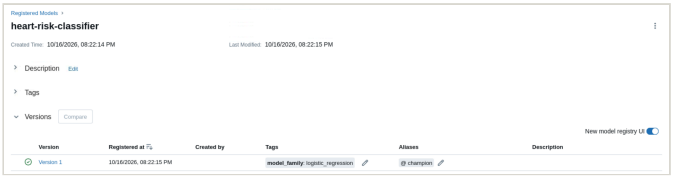


Figure 15. Model registry (excerpt of screenshot 11): version 1, alias @champion.

8. Model packaging and reproducibility

- **Artifact:** `artifacts/model/model.joblib` (single pipeline incl. preprocessing, SHA-256 a5b9c5de…) plus `model_metadata.json` (version `1.0.0+b00a9c9` = package version + git commit, feature list and order, threshold, best parameters, CV and test metrics, MLflow run id, data hash). The MLflow-format model is also stored and loadable via `models:/heart-risk-classifier@champion`.
- **Round-trip guarantee:** training reloads the saved file and asserts identical probabilities; tests check reload consistency, feature order, imputation of missing `ca`/`thal`, and unknown categories.
- **Environment:** all direct dependencies pinned (`requirements.txt`; slim `requirements-serve.txt` for the image, same versions). A real incompatibility (MLflow 2.22 with SQLAlchemy 2.1) was found and pinned. A fresh-environment install plus all tests was verified on Python 3.10, 3.11 and 3.12.
- **Determinism:** fixed seeds for split, CV folds and models; relative paths only; `.gitattributes` normalises line endings across Windows and Linux.

Independent re-execution	LR CV ROC-AUC	RF CV	GB CV	LR test ROC-AUC	RF test	GB test
Prayogshala VM, Python 3.12.9	0.907	0.901	0.876	0.959	0.960	0.953
GitHub Actions, Python 3.11 (run #4)	0.907	0.901	0.876	0.959	0.960	0.953

The identical results on two machines and Python versions demonstrate end-to-end reproducibility from the requirements file and the checksum-verified data.

9. CI/CD pipeline and automated testing

Tests (`test/`, Pytest): offline and deterministic — a synthetic fixture in the exact UCI raw format (including `?` values and duplicates) drives the data, feature and model tests, and a small grid trains real models with real MLflow logging in seconds. 53 tests, 95 % line coverage; executed in Prayogshala (all passed, 1 min 57 s) and on every CI run.

Module	Tests	What is verified
test_data.py	11	'?' parsing, schema/type/empty/missing-target rejection, duplicate and out-of-range removal, binarisation, no imputation before split
test_features.py	7	engineered-feature formula and toggle, no NaN after transform, scaling, unknown categories, imputer fitted on training data only (leakage guard)
test_model.py	11	metric correctness on a known case, seeding, selection rule, packaged round-trip, metadata, output structure, MLflow runs/artifacts/registry match returned metrics
test_api.py	18	health/readiness, prediction schema, request-id propagation, nullable fields, 422 for invalid/missing/unknown/malformed input, /metrics, logs contain no patient data, serving image imports no training stack
test_scripts.py	6	CLI entry points, EDA figures, download validation and ZIP fallback

Workflow (`.github/workflows/ci.yml`, on push and pull request to `main`; runner pinned to ubuntu-24.04; pip caching): **1 Lint** (flake8) → **2 Unit tests** (JUnit XML, coverage XML/HTML) → **3 Data, EDA and training** (download with checksum, clean, EDA, train with MLflow, inference check, metrics table written to the job summary) → **4 Docker** (build image with the trained model, start container, wait for readiness, assert a valid prediction and a 422 for invalid input, scrape `/metrics`). Each job uploads artifacts (lint report, test results, model, reports + MLflow store + logs, container logs). Every step runs in bash with `-eo pipefail` and jobs depend on each other, so **any failure stops the pipeline with a red status**; there is no error suppression.

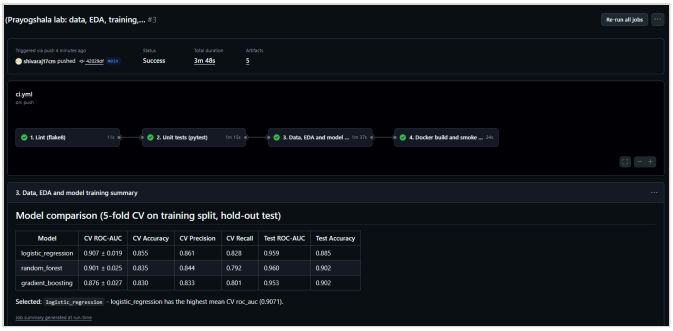


Figure 16. Successful run #3 (excerpt of screenshot 15): four dependent jobs and the metrics summary.

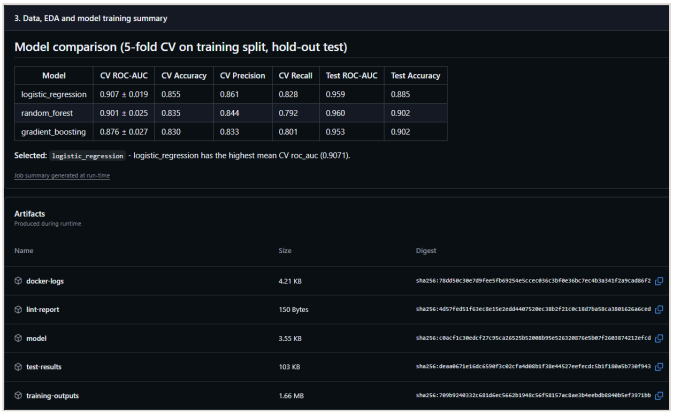


Figure 17. Run #4 (excerpt of screenshot 16): summary and five uploaded artifacts.

Proof that the pipeline fails on errors. A deliberately failing unit test was pushed on branch `demo/ci-failure` and opened as pull request #1: lint passed, unit tests failed with a clear assertion message and exit code 1, training and Docker jobs were skipped, the test report was still uploaded, and the PR was closed unmerged — `main` never received the broken code.

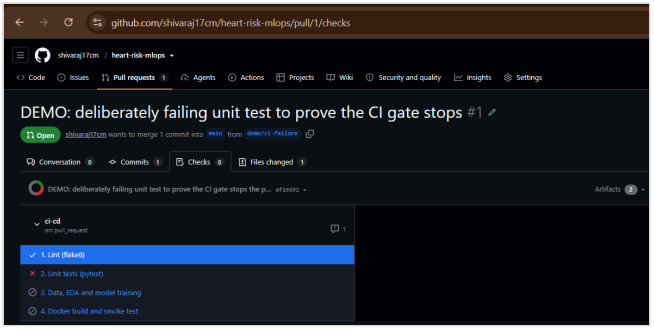


Figure 18. PR #1 checks: tests ×, downstream jobs skipped.

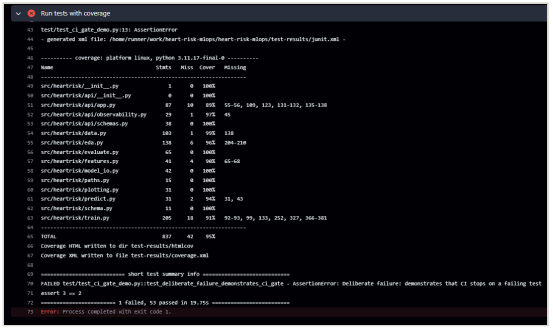


Figure 19. Failure log (excerpt of screenshot 17c): 1 failed, 53 passed, exit code 1.

10. Model containerisation and prediction API

Image design (`Dockerfile`): `python:3.11-slim`; dependencies installed first from `requirements-serve.txt` (cached layer; no MLflow/Jupyter in the image); application code and the trained pipeline copied in; runs as non-root numeric user `10001`; Docker `HEALTHCHECK` on `/ready`; port 8000. Image `heart-risk-api:1.0.0`, 163 MB content. A unit test guarantees the API imports none of the training stack, after an early run in a serving-only environment exposed exactly that defect.

API contract (FastAPI, OpenAPI at `/docs`): `POST /predict` accepts the 13 UCI attributes as JSON; numeric fields are range-checked, coded fields must be valid UCI codes, `ca`/`thal` may be `null` (imputed by the pipeline), unknown fields are rejected. The response carries the class, label, probability of disease, **confidence** (probability of the predicted class), model version and a request id. Invalid input returns HTTP 422 with a per-field error list and no stack trace. Other endpoints: `/health` (liveness), `/ready` (model loaded), `/metrics` (Prometheus).

```
POST /predict   (samples/sample_request.json)
{"age": 58, "sex": 1, "cp": 4, "trestbps": 132,
 "chol": 224, "fbs": 0, "restecg": 2,
 "thalach": 173, "exang": 0, "oldpeak": 3.2,
 "slope": 1, "ca": 2, "thal": 7}
```

```
200 OK  (actual response from the container)
{"prediction": 1,
 "label": "heart disease present",
 "probability_disease": 0.9615,
 "confidence": 0.9615,
 "model_version": "1.0.0+b00a9c9",
 "request_id": "2cb1ad6eeff04726"}
```

Build and test (screenshots 18–21): `docker build --no-cache -t heart-risk-api:1.0.0 .` completed all 7 steps; `docker run -d -p 8000:8000` reached *healthy*; the high-risk sample returned class 1 (0.9615) and the low-risk sample class 0 (probability 0.038, confidence 0.962) — exactly the values computed beforehand from the model file; an invalid payload returned 422.

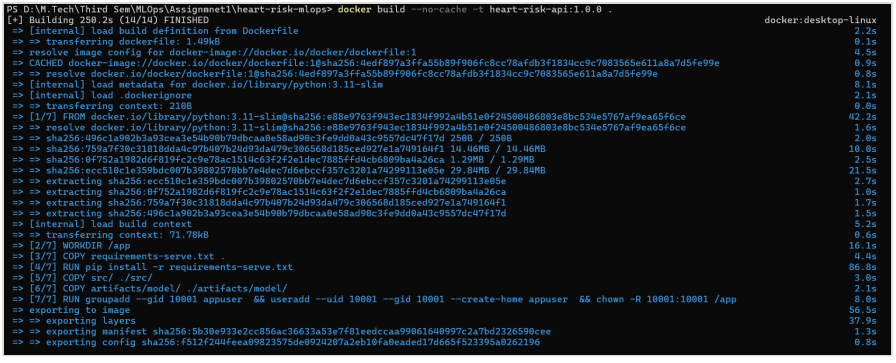


Figure 20. Clean image build (14/14 finished, 7 Dockerfile steps).

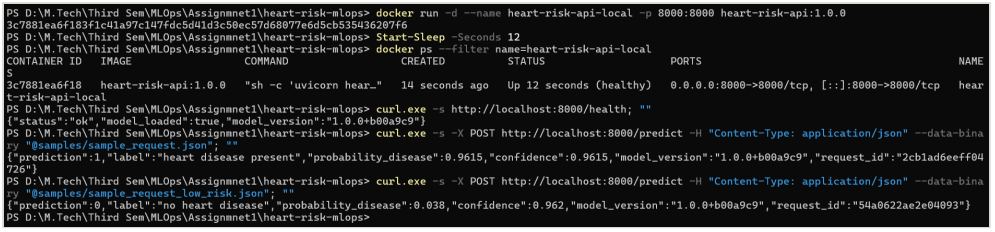


Figure 21. Running container: healthy status, /health and two real predictions.

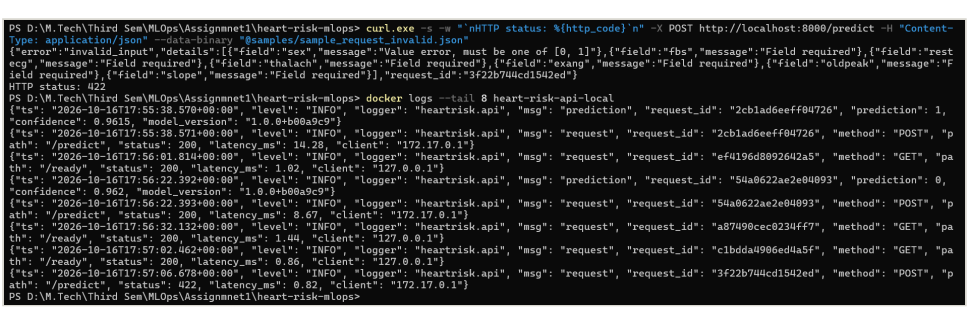


Figure 22. HTTP 422 for invalid input, and structured JSON container logs.

Incident handled. During the first local build the laptop's system drive ran out of space, which left a corrupt image layer ("unable to find user appuser"). Docker's virtual disk was moved to a larger drive, the image rebuilt with `--no-cache`, and the Dockerfile hardened to a numeric UID/GID so that Kubernetes `runAsNonRoot` can verify it; CI run #7 confirmed the change.

11. Production deployment on Kubernetes

Target. Local Kubernetes (Docker Desktop single-node cluster, v1.34.1), permitted by the assignment. All resources are declared in `deploy/` and applied with one command, `kubectl apply -k deploy/` (Kustomize generates the monitoring ConfigMaps from the same files docker-compose uses, a single source of truth). 15 resources are created in namespace `heart-risk`.

Manifest	Production concern addressed
10-api-deployment.yaml	2 replicas; RollingUpdate maxUnavailable 0 / maxSurge 1 (zero-downtime); startup, readiness (`/ready`) and liveness (`/health`) probes; requests 100m/192Mi, limits 500m/512Mi; runAsNonRoot uid 10001, read-only root FS, no privilege escalation, all capabilities dropped; Prometheus scrape annotations
11-api-service.yaml	Service type **LoadBalancer**, port 8080 → container 8000 (Docker Desktop publishes it on localhost; GKE/EKS/AKS would allocate a public IP). Optional NGINX Ingress in `deploy/k8s/optional/`
20-prometheus.yaml, 30-grafana.yaml	ServiceAccount + namespaced Role (least privilege: get/list/watch pods) for pod discovery; Grafana datasource and dashboard provisioned from ConfigMaps

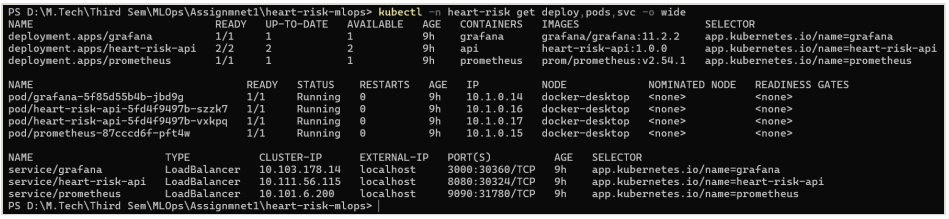


Figure 23. Deployments (API 2/2), four pods Running with 0 restarts, three LoadBalancer services.

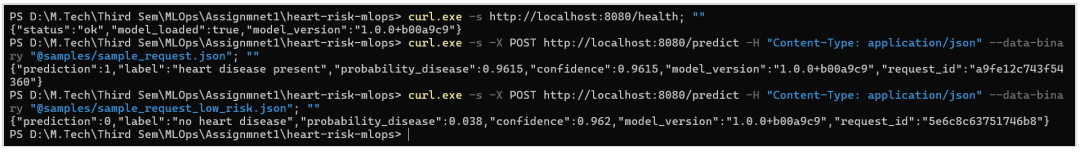


Figure 24. Real requests through the LoadBalancer (:8080) — identical predictions to the Docker run.

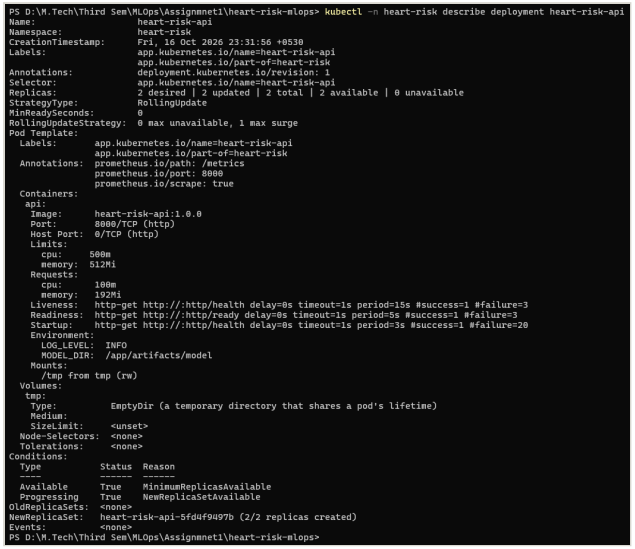


Figure 25. Deployment spec: replicas, strategy, resources, probes.

Verification performed

- `rollout status` succeeded for all three deployments (screenshots 22, 22b).
- Endpoint verified with `/health` and two `/predict` calls via the LoadBalancer.
- Self-healing observed: the Prometheus image pull first failed on transient network errors (short read, DNS) and Kubernetes retried with back-off until it succeeded, without intervention (screenshot 22c).

Docker run vs Kubernetes deployment

- Docker: one container, host port 8000, Docker health check, manual restart.
- Kubernetes: two replicas behind a load-balanced Service on 8080, readiness-gated traffic, automatic restarts and rolling updates, resource limits and pod-level security context, metrics discovered per pod.
- Same image (`heart-risk-api:1.0.0`), same model, same responses.

Access instructions

```
docker build -t heart-risk-api:1.0.0 .
kubectl apply -k deploy/
kubectl -n heart-risk rollout status \
  deployment/heart-risk-api
curl -X POST http://localhost:8080/predict \
  -H "Content-Type: application/json" \
  -d @samples/sample_request.json
```

12. Monitoring and logging

Logging: each request emits one JSON line (request id, method, route, status, latency, client) and each prediction one event (class, confidence, model version); the request id is returned as a response header for tracing. Patient feature values are never logged (enforced by a unit test; see screenshots 20 and 26). **Metrics** at `/metrics`: requests by route and status, latency histogram, predictions by class, confidence histogram, errors by reason, model-info gauge. Prometheus discovers both API pods via the Kubernetes API and Grafana shows a provisioned dashboard. A load test (`scripts/send_traffic.py`, 600 requests, ≈10 % deliberately invalid) plus verification calls produced 547 predictions and a 7.3 % error ratio. The error-ratio and latency panels detect regressions or overload, "pods up" detects crashes, and shifts in class mix or mean confidence (screenshot 28b) are an early signal of input drift.

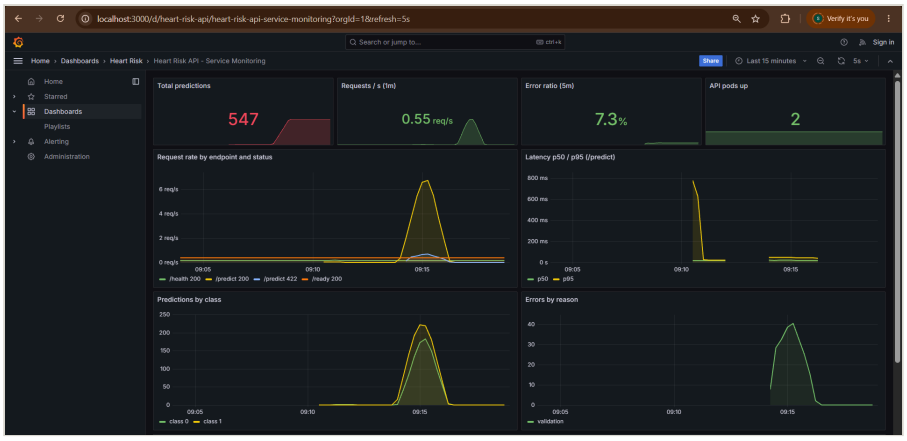


Figure 26. Grafana dashboard: predictions, request rate, error ratio, pods up, latency p50/p95, class mix, errors by reason.

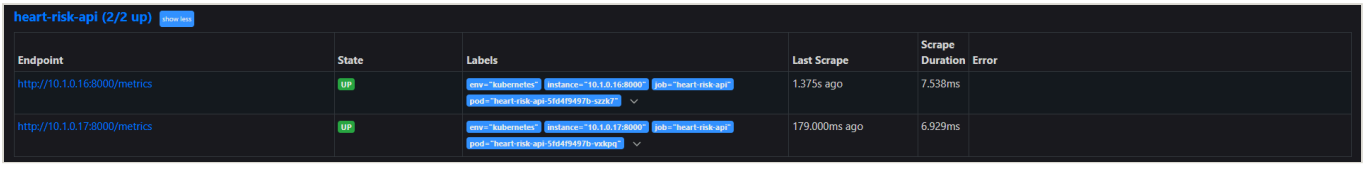


Figure 27. Prometheus targets (excerpt of screenshot 27): both API pods discovered and UP.

13. Results, limitations and conclusion

All nine assignment components were implemented, executed and evidenced: reproducible acquisition and EDA on real data; three tuned, cross-validated models with a pre-declared selection rule; complete MLflow tracking and registry; a single-object packaged pipeline reproduced identically on two platforms; 53 automated tests and a four-stage CI/CD pipeline that demonstrably blocks failures; a hardened container; a two-replica Kubernetes deployment; and Prometheus/Grafana monitoring. **Limitations:** 303 patients from a single centre limit generalisation; the 61-patient test set gives wide confidence intervals (test above CV is sampling variation); the 0.5 threshold is not tuned to clinical costs; deployment is local (cloud-ready manifests, no public endpoint); monitoring covers service health and prediction mix, not labelled-outcome drift. **Future work:** external validation on the other UCI sites, threshold calibration, an image registry with signed images, Helm packaging and alert rules.

14. Setup and reproduction

```
git clone https://github.com/shivaraj17cm/heart-risk-mlops.git && cd heart-risk-mlops
python -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\Activate.ps1
pip install -r requirements.txt                        # Python 3.10-3.12
bash scripts/run_pipeline.sh    # download, clean, EDA, train + MLflow, tests, inference
docker build -t heart-risk-api:1.0.0 . && kubectl apply -k deploy/   # serve + monitor
```

References

1. Janosi A., Steinbrunn W., Pfisterer M., Detrano R. (1989). *Heart Disease* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C52P4X
2. Detrano R. et al. (1989). International application of a new probability algorithm for the diagnosis of coronary artery disease. *Am. J. Cardiology*, 64(5), 304–310.
3. Pedregosa F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.
4. Sculley D. et al. (2015). Hidden Technical Debt in Machine Learning Systems. *NeurIPS 28*.
5. Fox S. M., Naughton J. P., Haskell W. L. (1971). Physical activity and the prevention of coronary heart disease. *Annals of Clinical Research*, 3, 404–432.
6. Documentation: MLflow (mlflow.org/docs), FastAPI (fastapi.tiangolo.com), Kubernetes (kubernetes.io/docs), Prometheus (prometheus.io/docs), Grafana (grafana.com/docs), GitHub Actions (docs.github.com/actions).